AI Interview Assistant API

Updated System Design Document

Node.js · Express.js · MongoDB · JWT · Gemini API · Multer

Revision focus: clarified service/data-access boundaries • explicit PDF extraction pipeline • 1:1 Question–Response • retained derived analytics • explicit RBAC • separate Resume Analysis flow

Document Revision Summary

- The architecture keeps the layered MVC structure but clarifies that the Data Access layer is implemented with Mongoose models and MongoDB aggregation; a separate repository layer is not required for the current scope.
- Gemini access is centralized through a reusable Gemini client/helper invoked by the service layer.
- Resume processing is explicitly modeled as Multer upload → PDF validation → PDF text extraction → ResumeAnalysis service → Gemini → MongoDB.
- Question → Response is defined as 1:1 because each question has exactly one submitted answer in the current interview flow.
- InterviewAnalytics remains a rolling, derived summary per candidate and is recomputed/updated after completed sessions.
- Role-based access control remains part of the design because the User model and API include candidate/admin roles.
- A separate Resume Analysis sequence is documented in addition to the main interview-session sequence.

1. Project Overview

The AI Interview Assistant API is a backend platform that simulates realistic, role-specific technical interviews. It uses the Gemini API to generate interview questions, evaluate candidate answers, and produce actionable feedback, scores, and improvement suggestions. It also analyzes uploaded resumes to surface skill gaps, ATS-readiness issues, and personalized learning recommendations.

The system is built with a modular MVC architecture on Node.js and Express.js, backed by MongoDB, and secured with JWT-based authentication, refresh-token support, bcrypt password hashing, and role-based access control.

1.1 Goals

- Generate role- and difficulty-specific interview questions on demand.
- Evaluate free-text candidate answers and return scores, feedback, and improvement tips.
- Let candidates upload a resume (PDF) and receive an AI-driven skill-gap / ATS analysis.
- Track interview history and analytics per user over time.
- Provide secure, role-based access to candidate and admin functionality through a clean REST API.

1.2 Tech Stack

Layer	Technology	Purpose
Runtime / Framework	Node.js, Express.js	HTTP server, routing, middleware pipeline
Database	MongoDB + Mongoose	Users, sessions, Q&A, resume analyses, analytics
AI Provider	Gemini API	Question generation, answer evaluation, resume analysis
AI Client	Reusable Gemini client/helper	Centralized provider configuration and API calls
Auth	JWT, bcrypt	Access/refresh tokens, password hashing, RBAC
File Uploads	Multer + PDF parser	PDF upload validation and text extraction
API Style	REST	Resource-oriented HTTP endpoints with JSON responses

2. System Architecture

The application follows a layered MVC architecture: incoming requests pass through middleware (authentication, authorization, validation, and file handling), reach thin route controllers, delegate business logic to services, and persist data through Mongoose models. Gemini calls are isolated behind a reusable Gemini client/helper so controllers remain free of

provider-specific logic.

Figure 1 — Updated Layered System Architecture

CLIENT LAYER
Web / Mobile Client — REST calls over HTTPS

↓

MIDDLEWARE
JWT authentication • Role checks • Validation • Multer • Centralized error handler

↓

ROUTES / CONTROLLERS
Map HTTP methods + paths • parse request • call service • shape response

↓

SERVICE LAYER
AuthService • InterviewService • EvaluationService • ResumeAnalysisService • AnalyticsService

↓

DATA ACCESS
Mongoose Models • Queries • Aggregation Pipelines
(No separate repository layer required for current scope)

↓

MONGODB ATLAS
Users • InterviewSessions • Questions • Responses • ResumeAnalyses • InterviewAnalytics

AI path: Service Layer → Gemini Client/Helper → Gemini API

2.1 Layer Responsibilities

Layer	Responsibility
Middleware	JWT verification, role checks, request validation, Multer parsing, centralized error handling
Routes / Controllers	Map HTTP verbs + paths to handlers; parse request, call service, shape response
Service Layer	Core business logic; build prompts; evaluate answers; orchestrate resume analysis; update analytics
Data Access	Mongoose schemas/models, queries, indexes, and MongoDB aggregation pipelines
Gemini Client	Centralized Gemini API configuration, request execution, timeout/retry behavior, provider isolation
PDF Parser	Extract text from validated PDF files after Multer handles upload

2.2 Cross-Cutting Concerns

- Centralized error handler with a custom AppError class and async-handler wrapper around controllers.
- Environment-based configuration via dotenv for development, staging, and production.
- Input validation using Joi or express-validator before service logic runs.
- Security middleware: helmet, CORS allow-list, and rate limiting on authentication and AI-calling routes.
- Structured logging using morgan for access logs and winston for application logs.
- Swagger / OpenAPI documentation generated from route and schema annotations.

3. Database Design — MongoDB Data Model

MongoDB is schema-flexible, but the collections below use fixed Mongoose schemas so relationships remain explicit and queryable. ObjectId references are used where relationships are needed; .populate() can be used when joined document data is required.

Figure 2 — Updated Collection Relationship Model

USER
_id • email • passwordHash • role • targetRole

↓

INTERVIEW SESSION (1:N)
userId • role • difficulty • status • overallScore

↓

QUESTION (1:N)
sessionId • text • type • topic • order

↓

RESPONSE (1:1)
questionId • sessionId • answerText • score • aiFeedback

USER (1:N) → RESUME ANALYSIS
userId • extractedSkills • skillGaps • atsScore • recommendations

USER (1:1) → INTERVIEW ANALYTICS
userId • avgScoreByTopic • progressTrend
Derived/aggregated; updated after completed sessions

3.1 Collections

Collection	Key Fields	Notes
User	email, passwordHash, role, targetRole	role is candidate or admin; drives route access
InterviewSession	userId, role, difficulty, status, overallScore	One document per interview attempt
Question	sessionId, text, type, topic, order	Generated by Gemini and scoped to one session
Response	questionId, sessionId, answerText, score, aiFeedback	Exactly one submitted answer per question in the current flow
ResumeAnalysis	userId, extractedSkills, skillGaps, atsScore, recommendations	One document per resume upload/analysis run
InterviewAnalytics	userId, avgScoreByTopic, progressTrend	One rolling summary per candidate; derived from completed sessions

3.2 Relationships and Indexes

- User 1—N InterviewSession: a candidate can take many interviews.
- InterviewSession 1—N Question: each session generates its own question set.
- Question 1—1 Response: each question gets exactly one submitted answer in the current interview flow.
- User 1—N ResumeAnalysis: a candidate may upload and analyze multiple resume versions.
- User 1—1 InterviewAnalytics: one rolling analytics summary is maintained per candidate.
- Indexes: User.email (unique); InterviewSession.userId; Question.sessionId; Response.sessionId + questionId; ResumeAnalysis.userId.

4. Business Logic — How It Works

The system has two major workflows: the interview-session lifecycle and the resume-analysis lifecycle. Both use the service layer for business logic and the centralized Gemini client for AI-provider access.

Figure 3 — Interview Session Sequence Flow

Actor / Layer	Action
CLIENT	POST /auth/login
AUTH MIDDLEWARE	Verify bcrypt password → issue access + refresh tokens
CLIENT	POST /interviews {role, difficulty}
INTERVIEW SERVICE	Create session (in-progress) → request questions
GEMINI CLIENT	Send role/difficulty prompt → receive structured questions

Actor / Layer	Action
DATA ACCESS	Persist Question documents linked to session
CLIENT	Receive sessionId + first question
CLIENT	POST /responses {questionId, answerText}
EVALUATION SERVICE	Send question + answer to Gemini
GEMINI CLIENT	Return score + feedback + strengths/gaps
DATA ACCESS	Persist Response; update session state
CLIENT	Receive feedback + next question
CLIENT	POST /interviews/:id/complete
ANALYTICS SERVICE	Compute overall score → update InterviewAnalytics

4.1 Authentication & Authorization

- User registers or logs in; password is compared against a bcrypt hash and is never stored in plain text.
- On success, the API issues a short-lived JWT access token and a longer-lived refresh token.
- Protected routes verify the JWT and attach the user identity and role to the request.
- Role-check middleware gates admin-only endpoints such as user management.

4.2 Interview Session Lifecycle

- Client starts a session using POST /interviews with target role and difficulty.
- InterviewService creates an InterviewSession with status = in-progress and asks Gemini for role-specific questions.
- Returned questions are persisted as Question documents linked to the session.
- For each answer, EvaluationService sends the question and answer to Gemini and saves the resulting score, feedback, strengths/gaps, and improvement suggestions as a Response.
- The flow repeats until all questions are answered. POST /interviews/:id/complete marks the session completed and computes the overall score.
- AnalyticsService then updates or recomputes the candidate's rolling InterviewAnalytics document.

4.3 Resume Analysis

The resume workflow explicitly separates upload handling from text extraction. Multer accepts and validates the multipart file, while a PDF parser extracts text before the ResumeAnalysisService calls Gemini.

CLIENT
POST /resume/upload (multipart/form-data)

↓

MULTER + VALIDATION
Check PDF MIME type / extension / size

↓

PDF PARSER
Extract raw text from the validated PDF

↓

RESUME ANALYSIS SERVICE
Build role-aware analysis prompt

↓

GEMINI CLIENT → GEMINI API
Skills • skill gaps • ATS issues • learning recommendations

↓

DATA ACCESS → RESUME ANALYSIS
Persist structured result in MongoDB

↓

CLIENT
Return structured analysis

4.4 Analytics

- After each completed session, AnalyticsService updates or recomputes InterviewAnalytics.
- Analytics tracks average score by topic and progress trend over time.
- MongoDB aggregation pipelines may be used to derive the rolling summary from completed sessions and responses.
- The separate analytics document is retained because the design intentionally provides a fast, user-facing summary rather than recalculating all history for every analytics request.

5. REST API Design

Method & Path	Purpose	Access
POST /auth/register	Create a new user account	Public
POST /auth/login	Authenticate and issue JWT tokens	Public
POST /auth/refresh	Exchange refresh token for a new access token	Authenticated
POST /interviews	Start interview session and generate questions	Candidate
GET /interviews/:id	Get session detail and question list	Candidate / Owner
POST /interviews/:id/complete	Mark session complete and compute overall score	Candidate / Owner
POST /responses	Submit answer for AI evaluation and feedback	Candidate
POST /resume/upload	Upload validated PDF and run AI analysis	Candidate
GET /resume/:id	Retrieve a past resume analysis	Candidate / Owner
GET /analytics/me	Get aggregated performance analytics	Candidate
GET /admin/users	List/manage users	Admin

5.1 API Behavior Notes

- Controllers should remain thin: validate/normalize input, call a service, and shape the HTTP response.
- Services should own interview, evaluation, resume, authentication, and analytics business rules.
- AI-provider-specific code should remain inside the Gemini client/helper rather than controllers.
- Owner checks should prevent a candidate from retrieving or completing another candidate's interview or resume analysis.

6. Non-Functional Considerations

6.1 Security

- Passwords hashed with bcrypt and never stored or logged in plain text.
- JWT access + refresh token strategy with short expiry on access tokens.
- Role-based access control enforced in middleware, not controller logic.
- Input validation and sanitization on every route, including upload MIME-type and size checks in Multer.
- Secrets and API keys loaded from environment variables and never hard-coded.

6.2 Reliability & Error Handling

- Async controllers are wrapped so unhandled promise rejections reach the centralized error handler.
- Custom AppError distinguishes operational errors such as bad input/not found from programming errors.
- Gemini calls use timeout + retry/backoff and return a clear fallback error if the AI provider is unavailable.
- Expected error cases include invalid JWT (401), invalid input/file (400), forbidden role access (403), missing resources (404), and provider/service failures (5xx as appropriate).

6.3 Scalability & Maintainability

- Stateless API with JWT-based authentication can be horizontally scaled behind a load balancer.

- Modular routes/controllers/services/models structure keeps AI-provider logic isolated and swappable.
- MongoDB indexes on frequently queried reference fields keep session and history lookups efficient.
- The current project intentionally avoids unnecessary repository/abstraction layers; a repository layer can be introduced later if the data-access complexity grows.

7. Future Enhancements

- Real-time interview mode over WebSockets with live follow-up questions based on previous answers.
- Voice input with speech-to-text before sending answers for evaluation.
- Multi-provider AI abstraction to swap or fall back between Gemini and other LLM providers.
- Admin dashboard for question-bank curation and platform-wide analytics.
- Caching layer such as Redis for frequently regenerated question sets by role/difficulty.

8. Implementation Structure

A practical project structure matching this design:

```
src/
■■■ routes/
■■■ controllers/
■■■ services/
■■■ models/
■■■ middleware/
■■■ utils/ (Gemini client, AppError, helpers)
■■■ validators/
■■■ config/
■■■ app.js / server.js
```

The PDF parser can live in a utility/service module, while Multer remains middleware. If the application grows substantially, repositories can be introduced under a dedicated data-access layer.

9. Design Decisions Finalized

- Use “MongoDB Data Model” rather than “ER Model” in documentation.
- Keep Question → Response as 1:1 for the current single-attempt-per-question interview flow.
- Keep sessionId on Response because session-level response queries are common and the index supports them.
- Keep InterviewAnalytics as a derived rolling summary because analytics are a first-class user-facing feature.
- Keep RBAC because the design includes candidate/admin roles and an admin endpoint.
- Explicitly model PDF parsing after Multer; Multer is upload handling, not PDF text extraction.
- Use a centralized Gemini client/helper to isolate provider-specific logic while keeping business decisions in services.

10. Summary

The updated design keeps the original project's strengths while removing ambiguity between the diagrams, database model, business logic, and implementation plan. The resulting architecture is intentionally modular without being over-engineered: controllers remain thin, services own business logic, Mongoose handles data access, Gemini is isolated behind a reusable client, and resume analysis has an explicit upload-to-AI pipeline.